COMPARATIVE ANALYSIS

# Backpropagation and Predictive Coding

*A controlled notebook comparison for California housing-value regression*

Based on the recorded run in the project notebook | 14 October 2026

## Executive summary

This report compares two neural-network training approaches implemented in the project notebook: a conventional feedforward regressor trained by backpropagation and a predictive-coding-inspired model trained with explicit layer-energy terms and iterative hidden-state updates. Both predict the continuous target median_house_value from the same California housing dataset, use the same 13 transformed input features, and are evaluated on the same held-out test partition.

In the recorded test results, backpropagation performs slightly better: MAE is $38,755 versus $39,904, RMSE is $57,255 versus $57,637, and $R^2$ is 0.7498 versus 0.7465. The differences are small—about 3.0% in MAE, 0.7% in RMSE, and 0.0033 $R^2$ points—and the notebook uses one seeded split. The result therefore supports a narrow conclusion: under this specific implementation and run, backpropagation has the edge, but the evidence does not establish that it is generally superior.

The experiment is useful as an educational comparison of optimization procedures. It is not a definitive algorithm benchmark. In particular, the PC model uses target-conditioned latent-state updates during training, but its test-time forward method initializes states from the input and predicts without iterative target-conditioned inference. This distinction matters when interpreting what “predictive coding” means in this implementation.

### Question and scope

The practical question is whether explicitly minimizing layer-wise prediction energy and revising hidden states can match a standard multilayer perceptron on tabular housing regression. The analysis focuses on the model code and saved artifacts, reports the measured test metrics, and identifies design choices that limit causal or general claims.

# 1. Experimental design and model formulations

## Data and evaluation protocol

The notebook loads 20,640 records and predicts median_house_value. Eight numeric predictors (location, age, room, bedroom, population, household, and income measurements) and the categorical ocean_proximity variable produce 13 encoded input columns. Missing predictor values are imputed, numeric features are standardized, and the categorical feature is one-hot encoded. The outer split holds out 20% (4,128 records) for final testing; preprocessing is fitted on training data, not on the test set. The target is standardized for optimization using the training mean (about $207,195) and standard deviation (about $115,619), while reported test errors are converted back to the original price scale.

Both models use hidden widths of 64 and 32, tanh nonlinearities, mini-batches of 256, Adam with learning rate 0.001, and 60 epochs for final training. The architecture and data are therefore broadly aligned, though the PC method performs additional state-gradient calculations and has an extra inference-step hyperparameter.

## Backpropagation model

The baseline is a conventional multilayer perceptron. Given input x, it computes $h_1 = \tanh(W_1x + b_1)$, $h_2 = \tanh(W_2h_1 + b_2)$, then $\hat{y} = W_3h_2 + b_3$. Its training objective is the mean squared error across examples. Automatic differentiation applies the chain rule through the complete computational graph; Adam updates every model parameter to reduce that loss. The implementation has 3,009 trainable parameters.

$$L(\theta) = (1/n)\,\Sigma_i\,(f\theta(x_i) - y_i)^2$$

## Predictive-coding-inspired model

The second model defines latent states $z_1$ and $z_2$ and local predictions $\mu_1 = \tanh(W_1x)$, $\mu_2 = \tanh(W_2z_1)$, and $\mu_3 = W_3z_2$. Its energy is the sum of three half-squared residuals: $E_1 = \tfrac{1}{2}\|z_1-\mu_1\|^2$, $E_2 = \tfrac{1}{2}\|z_2-\mu_2\|^2$, and $E_3 = \tfrac{1}{2}\|y-\mu_3\|^2$, averaged across the batch. Starting from feedforward states, training makes 1, 3, 5, or 10 gradient-descent updates to $z_1$ and $z_2$, then updates weights and biases with Adam on the resulting total energy.

$$E(z; x,y) = E_1 + E_2 + E_3; \quad z_l \leftarrow z_l - \eta\,\partial E/\partial z_l$$

The PC sweep trains each candidate for 20 epochs on an inner training split and chooses the inference-step count with lowest validation RMSE. Ten steps wins this sweep. The final PC model is then trained for 60 epochs. The code's test-time forward pass uses input-initialized states and $W_3$ to predict; it does not use test labels or perform target-conditioned latent refinement.

# 2. Results

Table 1 summarizes the final evaluation metrics saved in metadata.json. MAE and RMSE are in the target's original units (the dataset labels these as median house values); lower is better. $R^2$ measures improvement over predicting the test-set mean; higher is better.

| Model | MAE | RMSE | $R^2$ |
|---|---|---|---|
| Backpropagation | $38,755.10 | $57,255.46 | 0.7498 |
| Predictive coding | $39,903.80 | $57,636.65 | 0.7465 |
| PC minus backprop | +$1,148.70 | +$381.19 | −0.00334 |

## What the differences mean

Backpropagation reduces MAE by about $1,149 per test example relative to PC (2.97% of the backprop MAE). Its RMSE is lower by about $381 (0.67%); because RMSE squares errors before averaging, this metric is more sensitive to large misses. $R^2$ differs by about 0.00334, or 0.334 percentage points. Both models explain roughly three quarters of the test-set variance under the $R^2$ definition, and both leave substantial prediction error. The observed margin is modest, especially for RMSE and $R^2$.

These figures do not show that the methods are statistically distinguishable. There are no repeated random splits, confidence intervals, or paired significance tests in the notebook. A different seed, split, or optimization run could narrow, erase, or reverse the small gap. The appropriate statement is descriptive: in this single recorded test evaluation, the baseline scores better on all three reported metrics.

## Validation sweep and computational cost

| PC state steps | Validation RMSE | Validation $R^2$ | 20-epoch time* |
|---|---|---|---|
| 1 | $62,067.04 | 0.7214 | 7.5 s |
| 3 | $61,930.24 | 0.7226 | 6.5 s |
| 5 | $61,862.10 | 0.7232 | 7.2 s |
| 10 | $61,578.93 | 0.7257 | 18.2 s |

**Reported single-run timings on the notebook's CPU environment; treat as indicative only.*

Validation RMSE declines steadily as steps increase over the tested candidates, with an improvement of about 0.79% from one to ten steps. The 10-step run takes about 2.4 times as long as the 1-step run in this sweep. This suggests an accuracy–compute tradeoff in the explored range, but the sweep tests only four choices and timing is noisy; it does not establish that ten is globally optimal or most efficient.

## Training traces

The backprop standardized training MSE falls from about 0.500 in epoch 1 to 0.226 in epoch 60. The PC total energy also declines over training. In the saved test-batch energy diagnostic, $E_1$ and $E_2$ print as zero to six decimal places and $E_3$ is approximately 0.18765, so the displayed total is almost entirely output residual. This follows naturally from initializing each state to its own layer prediction, which makes the hidden consistency residuals zero initially; exact printed zeros should not be read as evidence that hidden-state inference never changes the states.

# 3. Comparative interpretation

## Predictive performance

The baseline's modest advantage is plausible for this task. A tabular regressor with nonlinear hidden layers can exploit feature interactions directly, and standard backpropagation provides a well-established gradient path for minimizing MSE. The PC model is competitive, but its state updates introduce extra optimization dynamics: results depend on initialization, state learning rate (0.05), number of inference updates, and the energy weighting. With only one configuration per method and a small inference-step sweep, the experiment cannot identify which component explains the final metric difference.

The target distribution also deserves care. Values in this dataset reach a maximum of $500,001, indicating a ceiling/censoring effect in the commonly used California housing data. This can affect the error distribution and limits how literally the models' predictions should be interpreted for the most expensive locations. The notebook uses raw-scale MAE/RMSE and a random split; it does not separately report performance by price band, geography, or capped-target status.

## What the comparison controls—and what it does not

The design controls several important factors: both models share the same outer train/test partition, input features, preprocessing strategy, hidden widths, activation family, optimizer family, batch size, and final epoch count. The held-out targets are not used to tune the inference-step count. The PC sweep uses an inner validation partition taken from the outer training set, which is preferable to selecting hyperparameters on the test set.

However, equal epoch counts are not equal compute budgets. Every PC mini-batch performs multiple latent-state gradient updates in addition to parameter learning, while the backprop model performs its standard forward/backward update. A fair efficiency claim would compare wall-clock or operation budgets across repeated runs, not epoch counts alone. In addition, only PC inference steps receive an explicit validation sweep; the baseline's learning rate, width, regularization, and epoch count are not tuned through an equivalent search. This is a reasonable classroom experiment, but not a fully optimized head-to-head benchmark.

## Terminology and interpretation of "predictive coding"

The notebook explicitly implements a layered squared-error energy and gradient-based latent-state refinement. It is therefore most precise to call it a predictive-coding-inspired supervised model or this notebook's PC formulation. The results should not be generalized to every predictive-coding network or interpreted as a test of biological plausibility. The implementation uses automatic differentiation to obtain latent-state gradients and Adam to update parameters; those engineering choices are part of the method being evaluated.

One particularly important distinction is that training-state inference uses the known minibatch target y through $E_3$. At evaluation, y is unknown and the forward method uses only x to initialize $z_1$ and $z_2$ and then produce $\hat{y}$. Consequently, the test comparison is valid as supervised prediction, but it does not test a target-conditioned iterative settling procedure at inference time. Any report should make that distinction explicit.

# 4. Limitations, recommendations, and conclusion

## Limitations

The main limitation is statistical: one seeded 80/20 split gives a point estimate, not a reliable estimate of average performance. There are no repeated seeds, uncertainty intervals, or subgroup analyses. Small observed metric differences could be due to split or initialization variability. The report also relies on one recorded execution and environment; package versions, CPU/GPU behavior, and nondeterminism can affect exact reproducibility.

The comparison is also sensitive to methodological choices. The PC result depends on state initialization, the state-update step size, number of inference steps, and the relative scale of layer energies. The sweep evaluates four counts only. The experiment does not report a simpler linear or tree-based baseline, nor does it compare under matched compute. Finally, because the dataset includes a capped target and the test split is random rather than spatial, the scores do not establish performance under geographic or temporal distribution shift.

## Recommendations for a stronger follow-up

A stronger study should repeat the full train/validation/test process over multiple fixed seeds or use repeated cross-validation, then report mean and standard deviation (or confidence intervals) for MAE, RMSE, and $R^2$. Keep a final untouched test set if model selection is substantial. Tune both approaches under comparable validation budgets and compare both epochs and wall-clock compute. Include a simple mean/linear baseline and a strong tabular baseline to contextualize the neural models.

For the PC method, document the exact inference protocol, report state-update convergence and energy components at higher precision, and ablate inference steps, state learning rate, and layer-energy weights. If the intended claim concerns iterative predictive coding at inference, implement a clearly defined input-only inference objective that does not require the unknown target, then compare that protocol separately. Report metrics by target-price band and examine the capped-value subset. These additions would distinguish model quality, optimization quality, and computational cost.

## Conclusion

On the recorded California housing experiment, backpropagation is the best of the two evaluated configurations, but only narrowly: its MAE, RMSE, and $R^2$ are each slightly better than those of the predictive-coding-inspired model. The PC method's validation results improve as the tested inference-step count rises, at increased training time. The overall evidence supports the educational value of comparing global parameter-gradient training with explicit latent-state energy minimization, while remaining insufficient for broad claims about either method's general superiority.

The most defensible takeaway is conditional: for this dataset, architecture, training setup, and single held-out split, standard backpropagation achieved slightly lower prediction error with a simpler optimization loop. More runs and matched compute are needed to know whether that margin is stable or whether a tuned PC formulation can close it.

## References and project materials

[1] Stenlund, M. (2025). Introduction to Predictive Coding Networks for Machine Learning. arXiv:2506.06332. Project copy: doc/2506.06332v1.pdf.

[2] Project notebook. Comparing Backpropagation and Predictive Coding for House-Value Regression. Executed experiment and saved outputs: notebooks/icoglabs_backprop_vs_predictive_coding_house_values.ipynb and notebooks/artifacts/house_value_regression/metadata.json.

[3] hosammhmdali. House Price Dataset. Kaggle dataset downloaded at notebook runtime; target median_house_value. Dataset copy is not committed in this repository.

*Prepared from the code and recorded outputs in the project repository. Values are reported as saved; interpretation is limited to the experiment actually run.*